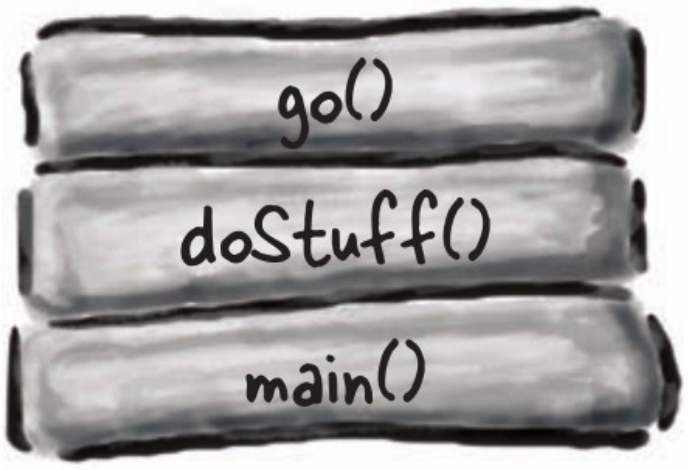
Life and Death of an Object

The Stack

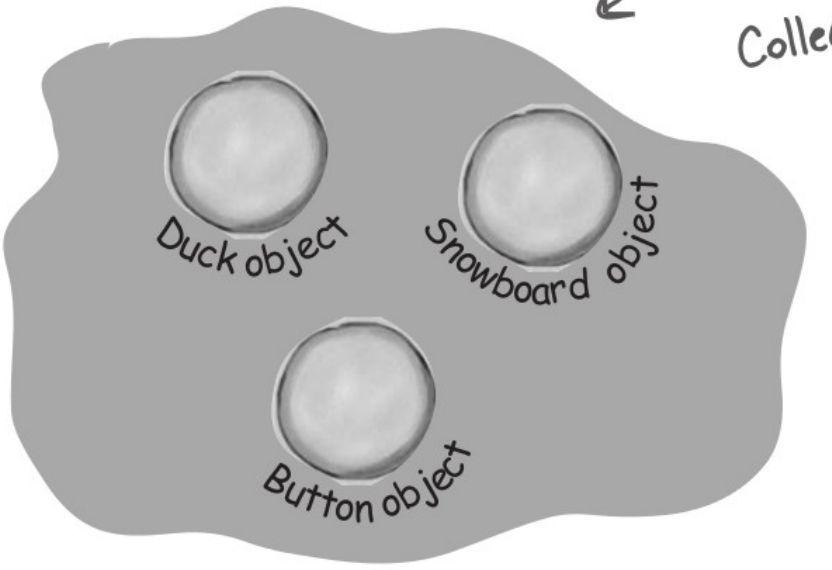
Where method invocations and local variables live



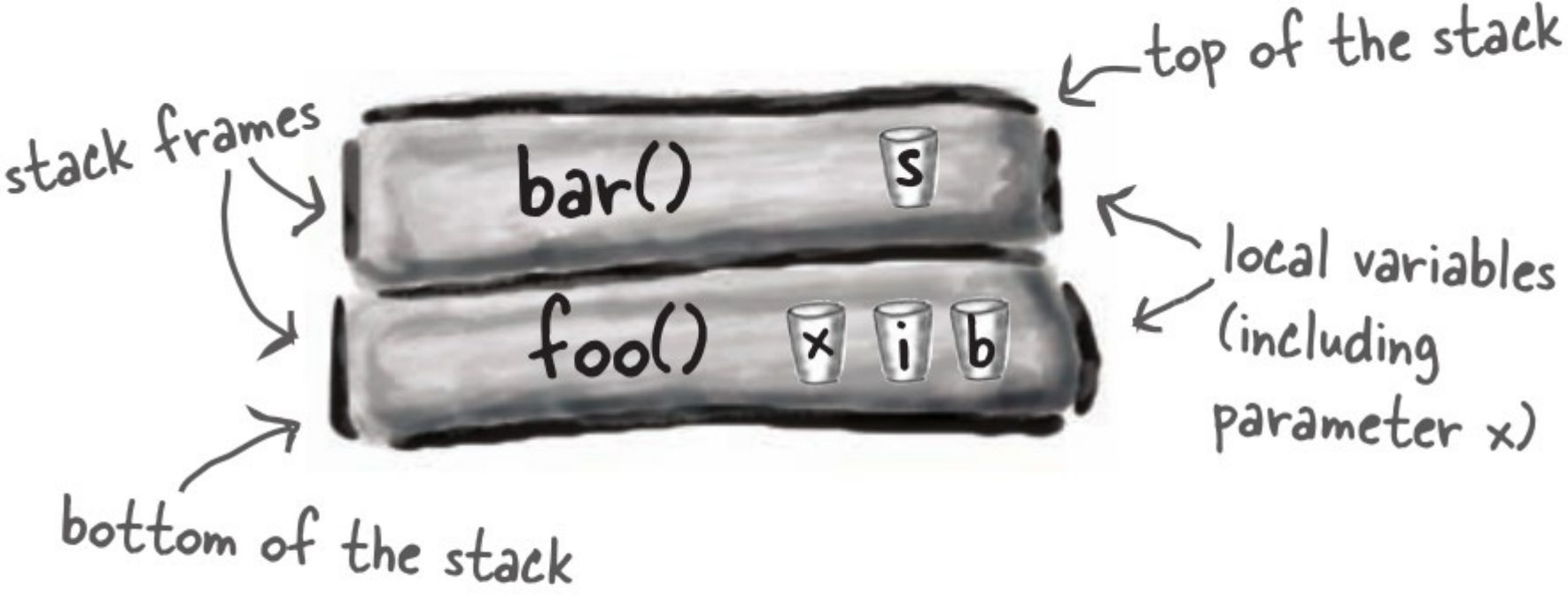
The Heap

Where ***ALL*** objects live

also known as "The Garbage-Collectible Heap"



A call stack with two methods



The method on the top of the stack is always the currently-executing method.

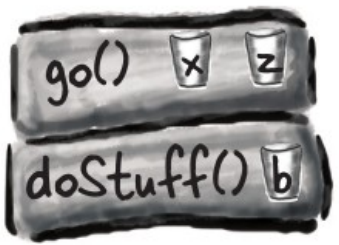
```
public void doStuff() {
  boolean b = true;
  go(4);
}
```

```
public void go(int x) {
  int z = x + 24;
  crazy();
  // imagine more code here
}
```

```
public void crazy() {
  char c = 'a';
}
```

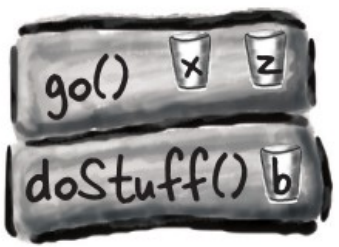
1. Code from another class calls **doStuff()**, and **doStuff()** goes into a stack frame at the top of the stack. The boolean variable named "**b**" goes on the **doStuff()** stack frame.

2. **doStuff()** calls **go()**, and **go()** is *pushed* on top of the stack. Variables "**x**" and "**z**" are in the **go()** stack frame.



3. **go()** calls **crazy()**, **crazy()** is now on the top of the stack, with variable "**c**" in the frame.

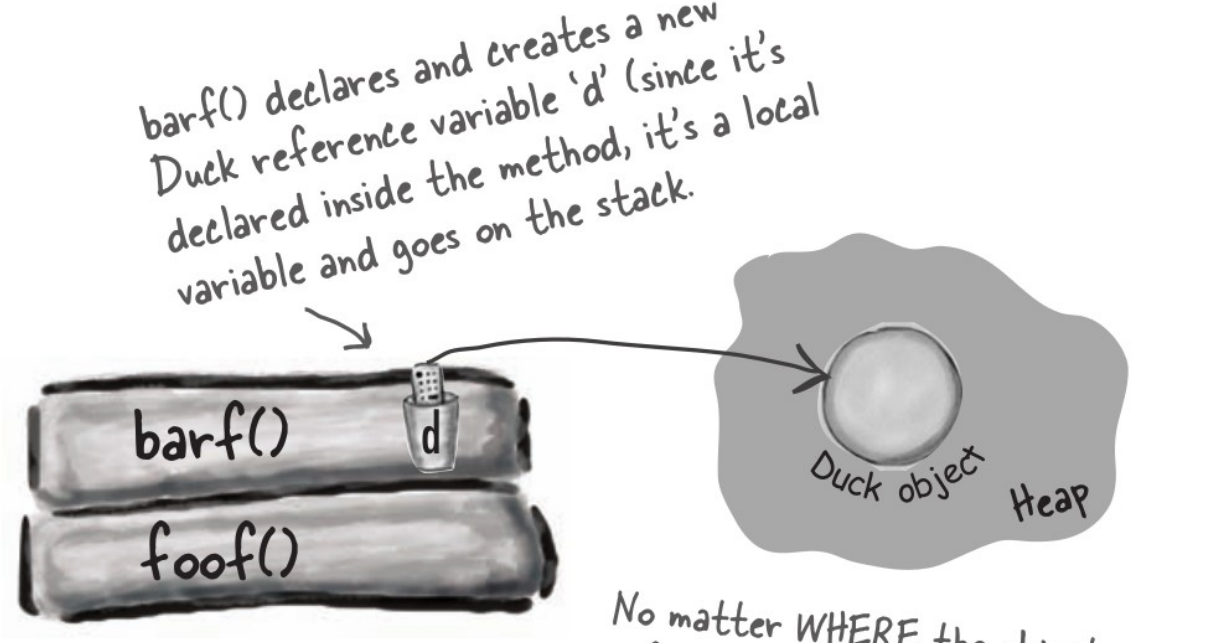
4. **crazy()** completes, and its stack frame is *popped* off the stack. Execution goes back to the **go()** method and picks up at the line following the call to **crazy()**.



What about local variables that are objects?

The object itself still goes in the heap.

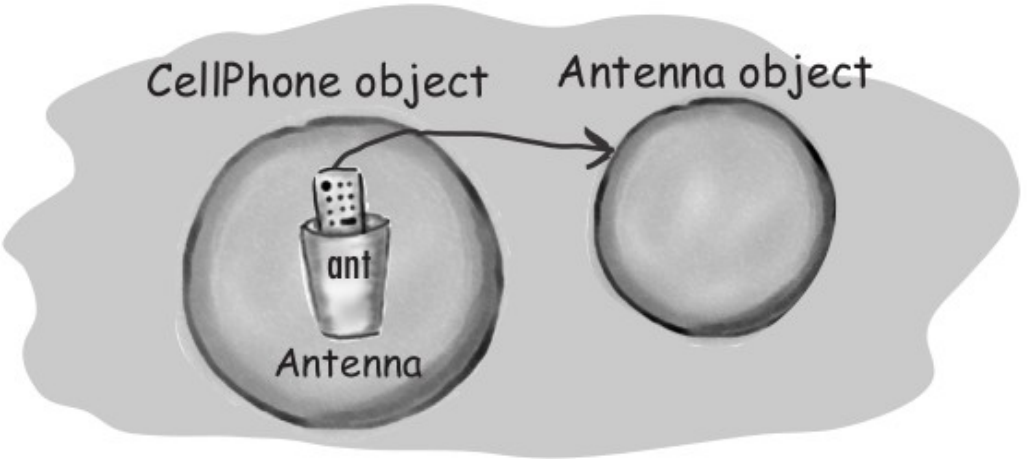
```
public class StackRef {
   public void foof() {
      barf();
   }

   public void barf() {
      Duck d = new Duck(24);
   }
}
```



If local variables live on the stack, where do instance variables live?

Object with one non-primitive instance variable—a reference to an Antenna object, but no actual Antenna object. This is what you get if you declare the variable but don't initialize it with an actual Antenna object.

```
public class CellPhone {
  private Antenna ant;
}
```



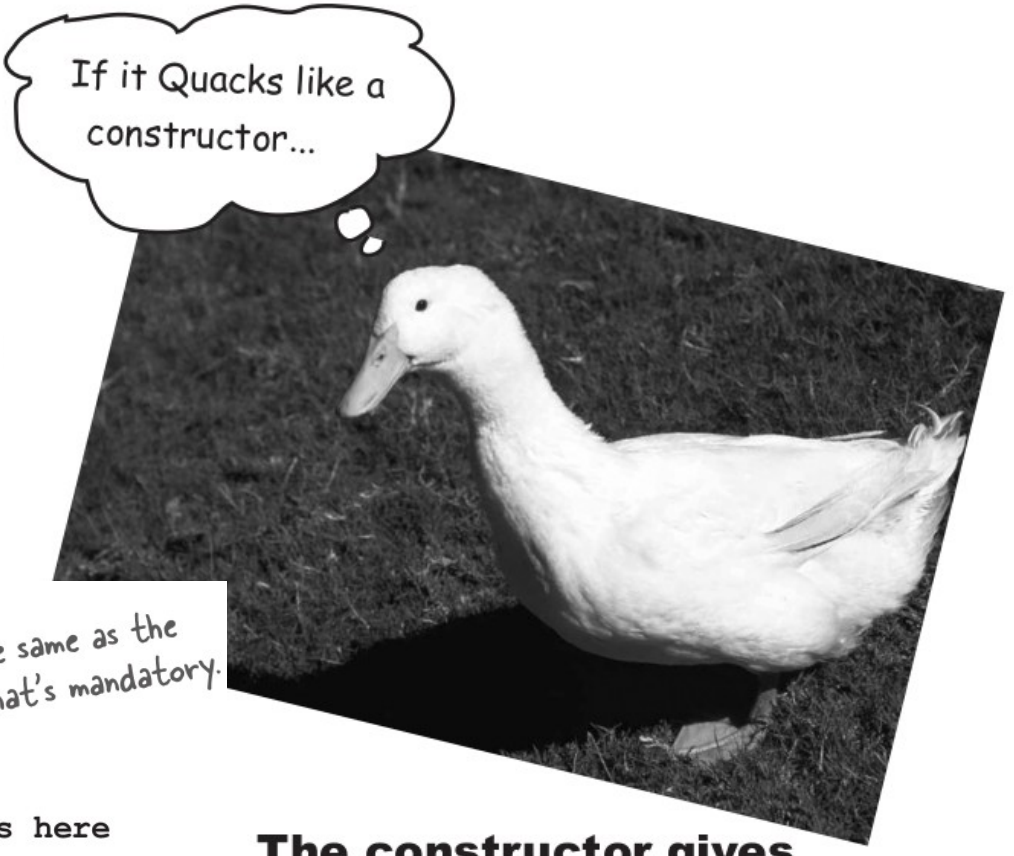
Object with one non-primitive instance variable, and the Antenna variable is assigned a new Antenna object.

```
public class CellPhone {
  private Antenna ant = new Antenna();
}
```

Construct a Duck

```
public class Duck {

    public Duck() {
        System.out.println("Quack");
    }
}
```

Constructor code.



Notice something missing? How is this different from a method?

Its name is the same as the class name. That's mandatory.

```
public Duck() {
    // constructor code goes here
}
```

Where's the return type? If this were a method, you'd need a return type between "public" and "Duck()".

The constructor gives you a chance to step into the middle of `new`.

```
public class Duck {
   int size;

   public Duck() {
      System.out.println("Quack");
   }

   public void setSize(int newSize) {
      size = newSize;
   }
}
```

instance variable

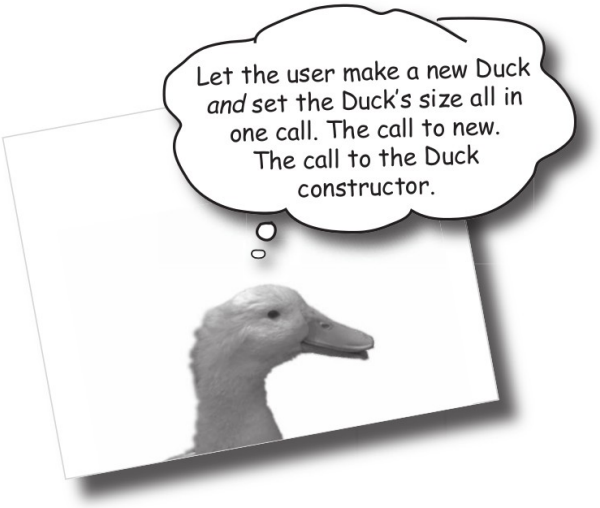
constructor

setter method

```
public class UseADuck {

   public static void main (String[] args){
      Duck d = new Duck();

      d.setSize(42);
   }
}
```

There's a bad thing here. The Duck is alive at this point in the code, but without a size!* And then you're relying on the Duck-user to KNOW that Duck creation is a two-part process: one to call the constructor and one to call the setter.

```
public class Duck {
  int size;

  public Duck(int duckSize) {
    System.out.println("Quack");

    size = duckSize;

    System.out.println("size is " + size);
  }
}
```



```
public class UseADuck {

  public static void main (String[] args) {
    Duck d = new Duck(42);
  }
}
```

This time there's only one statement. We make the new Duck and set its size in one statement.

Pass a value to the constructor.

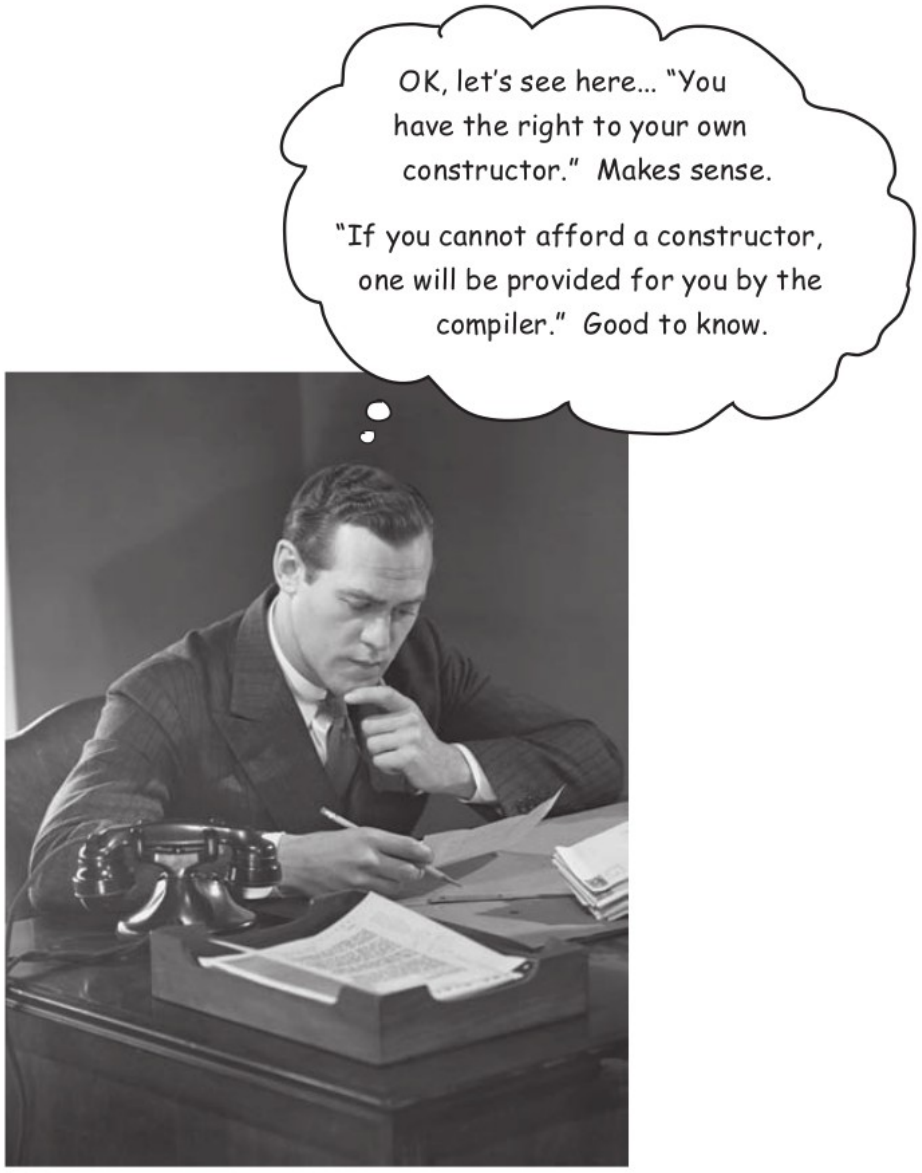
```
File Edit Window Help Honk
% java UseADuck
Quack
size is 42
```

```
public class Duck {
    int size;

    public Duck(int newSize) {
        if (newSize == 0) {
            size = 27;
        } else {
            size = newSize;
        }
    }
}
```

If the parameter value is zero, give the new Duck a default size, otherwise use the parameter value for the size. NOT a very good solution.

You really want TWO ways to make a new Duck:

```
public class Duck2 {
    int size;

    public Duck2() {
        // supply default size
        size = 27;
    }

    public Duck2(int duckSize) {
        // use duckSize parameter
        size = duckSize;
    }
}
```



Doesn't the compiler always make a no-arg constructor for you? No!

If you write a constructor that takes arguments and you *still* want a no-arg constructor, you'll have to build the no-arg constructor yourself!

If you have more than one constructor in a class, the constructors MUST have different argument lists.

Overloaded constructors means you have more than one constructor in your class.

To compile, each constructor must have a *different* argument list!

Five different constructors means five different ways to make a new mushroom.

```
public class Mushroom {

  public Mushroom(int size) { }

  public Mushroom( ) { }

  public Mushroom(boolean isMagic) { }

  public Mushroom(boolean isMagic, int size) { }

  public Mushroom(int size, boolean isMagic) { }
}
```

When you know the size, but you don't know if it's magic

When you don't know anything

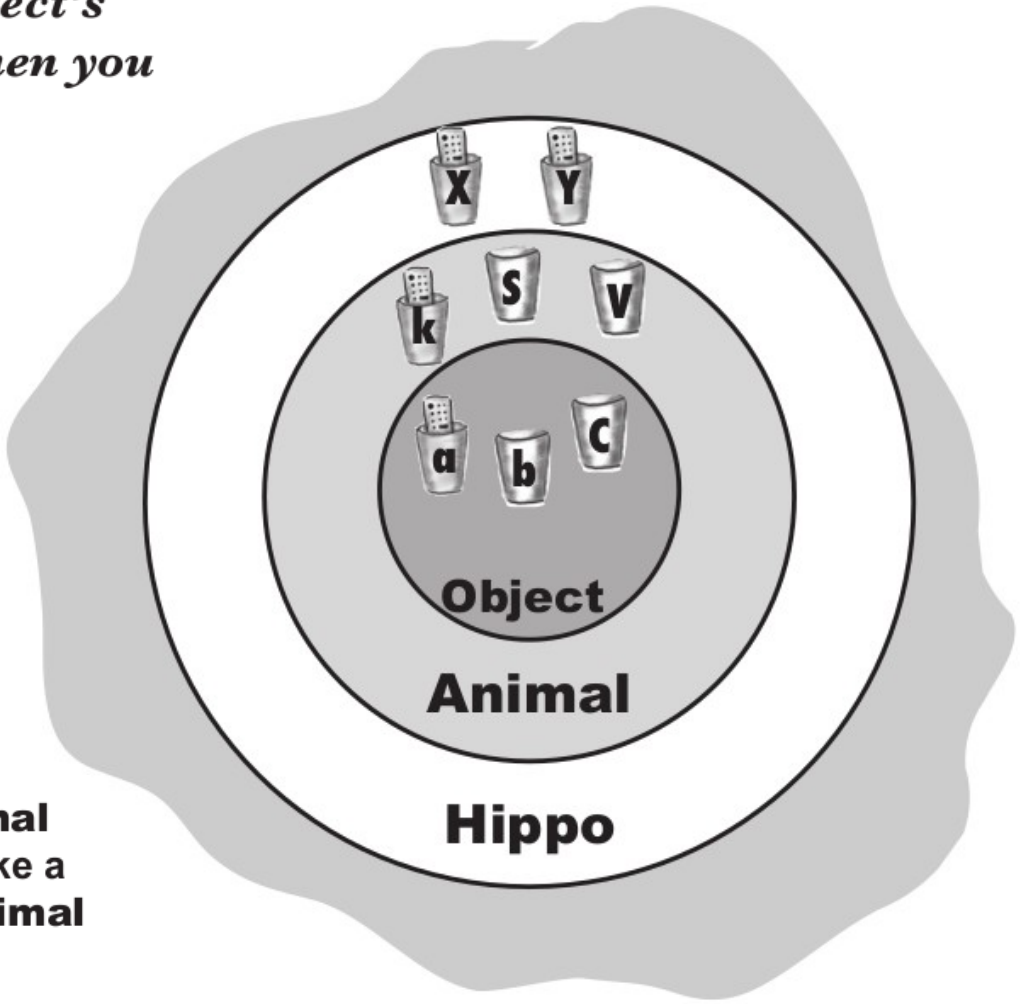
When you know if it's magic or not, but don't know the size

These two have the same args, but in a different order, so it's OK*

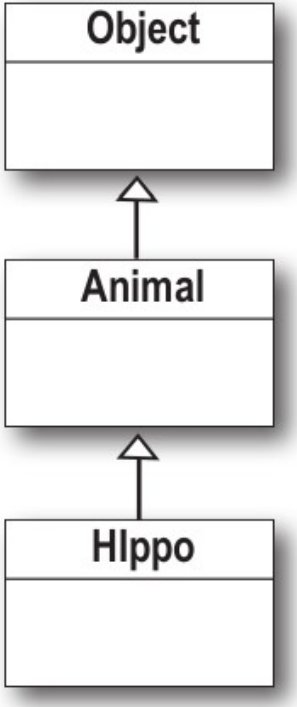
When you know whether or not it's magic, AND you know the size as well

*If the arguments were the same type, how would the compiler know they were two different things?

All the constructors in an object's inheritance tree must run when you make a new object.



A single Hippo object on the heap



A new **Hippo** object also IS-A **Animal** and IS-A **Object**. If you want to make a **Hippo**, you must also make the **Animal** and **Object** parts of the **Hippo**.

This all happens in a process called **Constructor Chaining**.

```
public class Animal {
   public Animal() {
      System.out.println("Making an Animal");
   }
}
```

```
public class Hippo extends Animal {
  public Hippo() {
    System.out.println("Making a Hippo");
  }
}
```

```
public class TestHippo {
  public static void main(String[] args) {
    System.out.println("Starting...");
    Hippo h = new Hippo();
  }
}
```

How do you invoke a superclass constructor?

```
public class Duck extends Animal {
   int size;

   public Duck(int newSize) {
      Animal();
      size = newSize;
   }
}
```

BAD! → `Animal();` ← NO! This is not legal!

1 If you *don't* provide a constructor

The compiler puts one in that looks like:

```
public ClassName() {
  super();
}
```

```
public class Duck extends Animal {
   int size;

   public Duck(int newSize) {
      super();
      size = newSize;
   }
}
```

`super();` ← You just say super()

2 If you *do* provide a constructor but you do *not* put in the call to super()

The compiler will put a call to super() in each of your overloaded constructors.* The compiler-supplied call looks like:

```
super();
```

Can the child exist before the parents?

The call to super() must be the *first* statement in each constructor!*

Possible constructors for class Boop

☑
```
public Boop() {
    super();
}
```

☑
```
public Boop(int i) {
    super();
    size = i;
}
```

These are OK because the programmer explicitly coded the call to super(), as the first statement.

☑
```
public Boop() {
}
```

☑
```
public Boop(int i) {
    size = i;
}
```

These are OK because the compiler will put a call to super() in as the first statement.

⊘
```
public Boop(int i) {
    size = i;
    super();
}
```

BAD!! This won't compile! You can't explicitly put the call to super() below anything else.

Ma funziona da Java 25

```
public abstract class Animal {
  private String name;

  public String getName() {
    return name;
  }

  public Animal(String theName) {
    name = theName;
  }
}
```

All animals (including subclasses) have a name.

A getter method that Hippo inherits.

The constructor that takes the name and assigns it the name instance variable.

```
public class Hippo extends Animal {
  public Hippo(String name) {
    super(name);
  }
}
```

Hippo constructor takes a name.

It sends the name up the Stack to the Animal constructor.

Use this() to call a constructor from another overloaded constructor in the same class.

The call to this() can be used only in a constructor, and must be the first statement in a constructor.

A constructor can have a call to super() OR this(), but never both!

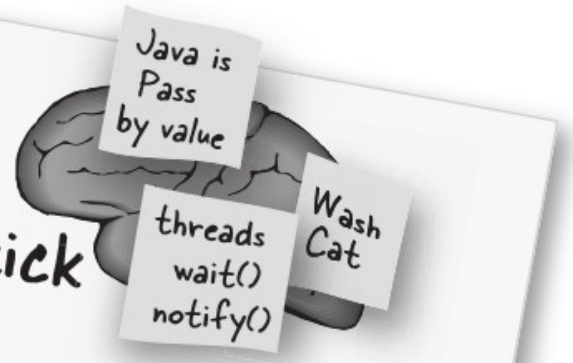
```
import java.awt.Color;

class Mini extends Car {
  private Color color;

  public Mini() {
    this(Color.RED);
  }

  public Mini(Color c) {
    super("Mini");
    color = c;
    // more initialization
  }

  public Mini(int size) {
    this(Color.RED);
    super(size);
  }
}
```

The no-arg constructor supplies a default Color and calls the overloaded Real Constructor (the one that calls super()).

This is The Real Constructor that does The Real Work of initializing the object (including the call to super()).

Won't work!! Can't have super() and this() in the same constructor, because they each must be the first statement!

Make it Stick



Roses are red, violets are blue.
Your parents come first, way before you.

The superclass parts of an object must be fully-formed before the new subclass object can exist. Just like there's no way *you* could have been born *before* your parents.

BE the compiler

```
public class Boo {
   public Boo(int i) { }
   public Boo(String s) { }
   public Boo(String s, int i) { }
}
```

```
class SonOfBoo extends Boo {

   public SonOfBoo() {
      super("boo");
   }

   public SonOfBoo(int i) {
      super("Fred");
   }

   public SonOfBoo(String s) {
      super(42);
   }

   public SonOfBoo(int i, String s) {
   }

   public SonOfBoo(String a, String b, String c) {
      super(a,b);
   }

   public SonOfBoo(int i, int j) {
      super("man", j);
   }

   public SonOfBoo(int i, int x, int y) {
      super(i, "star");
   }
}
```

BE the compiler

```
public class Boo {
   public Boo(int i) { }
   public Boo(String s) { }
   public Boo(String s, int i) { }
}
```

```
class SonOfBoo extends Boo {

   public SonOfBoo() {
      super("boo");
   }

   public SonOfBoo(int i) {
      super("Fred");
   }

   public SonOfBoo(String s) {
      super(42);
   }

   public SonOfBoo(int i, String s) {
   }

   public SonOfBoo(String a, String b, String c) {
      super(a,b);
   }

   public SonOfBoo(int i, int j) {
      super("man", j);
   }

   public SonOfBoo(int i, int x, int y) {
      super(i, "star");
   }
}
```

An object becomes eligible for GC when its last live reference disappears.

Three ways to get rid of an object's reference:

1. The reference goes out of scope, permanently

```
void go() {
  Life z = new Life();
}
```

reference 'z' dies at end of method.

2. The reference is assigned another object

```
Life z = new Life();
z = new Life();
```

the first object is abandoned when z is 'reprogrammed' to a new object.

3. The reference is explicitly set to null

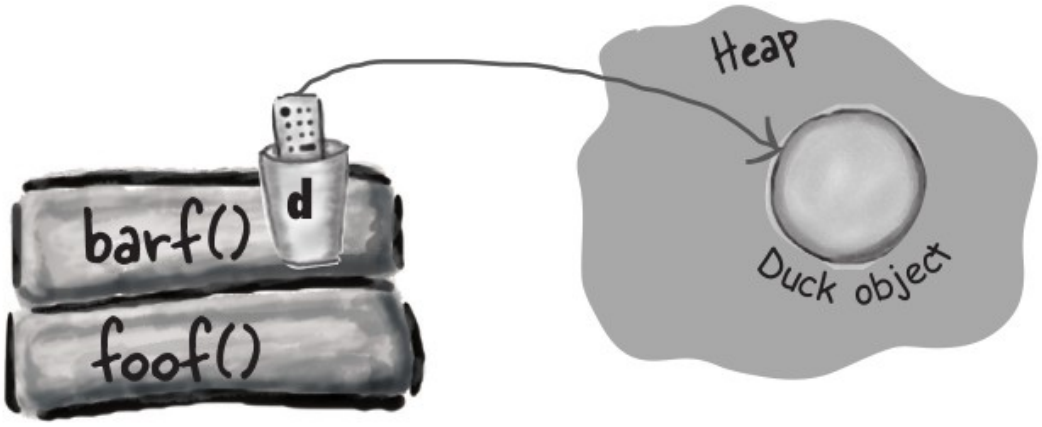
```
Life z = new Life();
z = null;
```

the first object is abandoned when z is 'deprogrammed.'

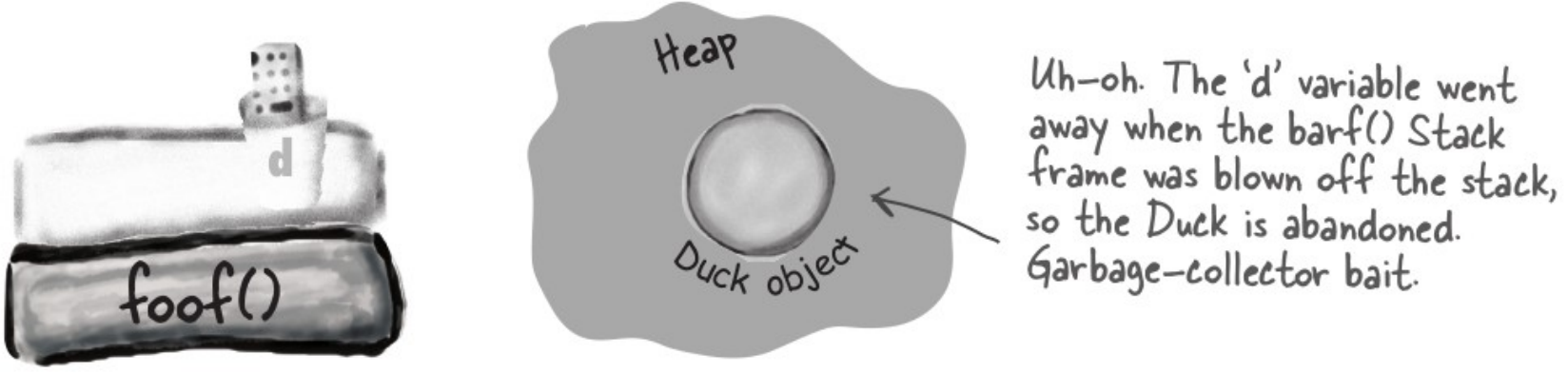
Object-killer #1

Reference goes out of scope, permanently.

```
public class StackRef  {
   public void foof() {
      barf();
   }

   public void barf() {
      Duck d = new Duck();
   }
}
```



The new Duck goes on the Heap, and as long as barf() is running, the 'd' reference is alive and in scope, so the Duck is considered alive.

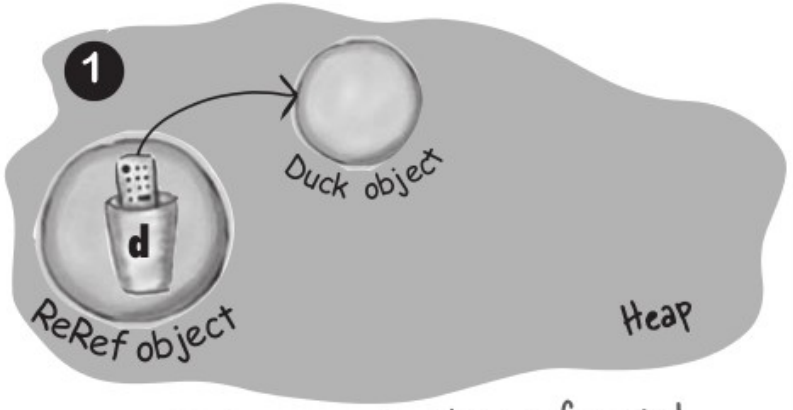


Uh-oh. The 'd' variable went away when the barf() Stack frame was blown off the stack, so the Duck is abandoned. Garbage-collector bait.

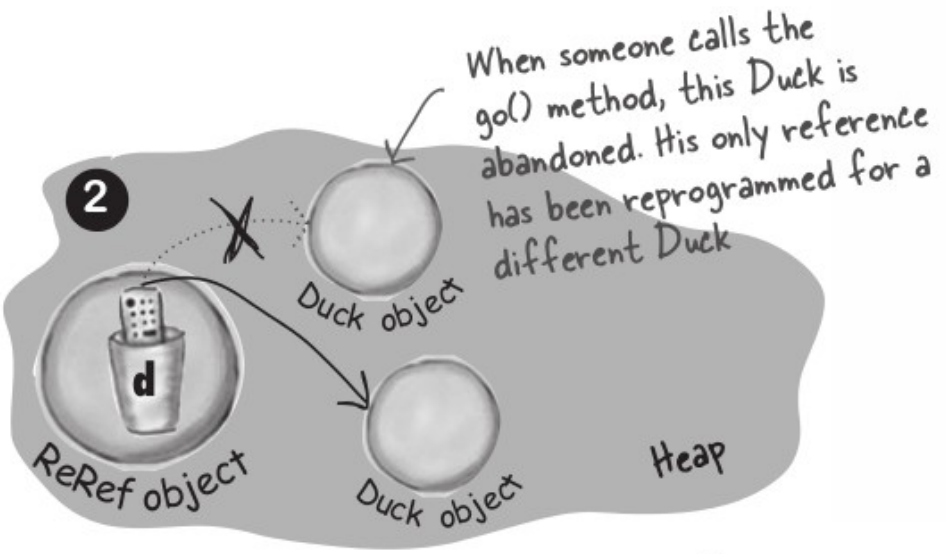
Object-killer #2

Assign the reference to another object

```
public class ReRef {

    Duck d = new Duck();

    public void go() {
      d = new Duck();
    }
}
```



The new Duck goes on the Heap, referenced by 'd'. Since 'd' is an instance variable, the Duck will live as long as the ReRef object that instantiated it is alive. Unless...

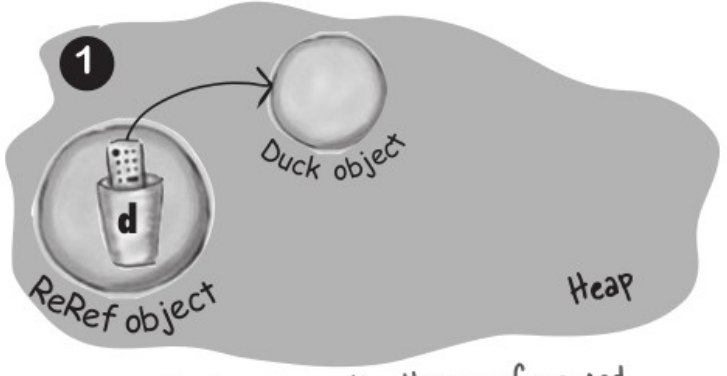


'd' is assigned a new Duck object, leaving the original (first) Duck object abandoned. That first Duck is now as good as dead.

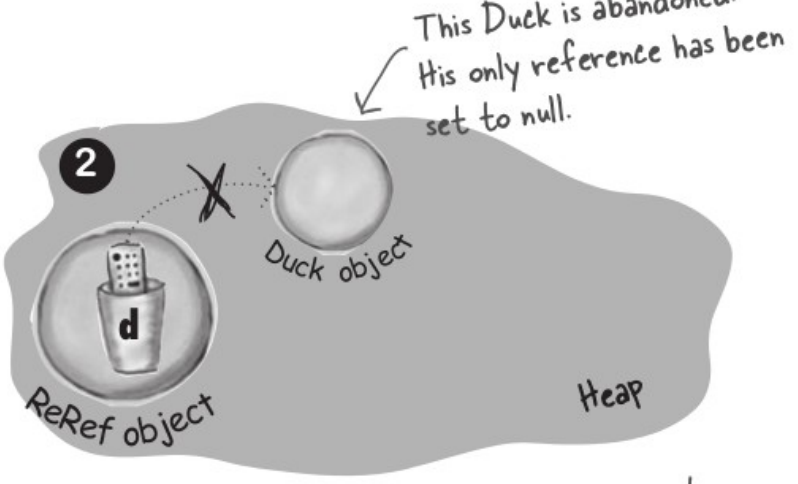
Object-killer #3

Explicitly set the reference to null

```
public class ReRef {
  Duck d = new Duck();

  public void go() {
    d = null;
  }
}
```



The new Duck goes on the Heap, referenced by 'd'. Since 'd' is an instance variable, the Duck will live as long as the ReRef object that instantiated it is alive. Unless...



'd' is set to null, which is just like having a remote control that isn't programmed to anything. You're not even allowed to use the dot operator on 'd' until it's reprogrammed (assigned an object).

Life on the garbage-collectible heap

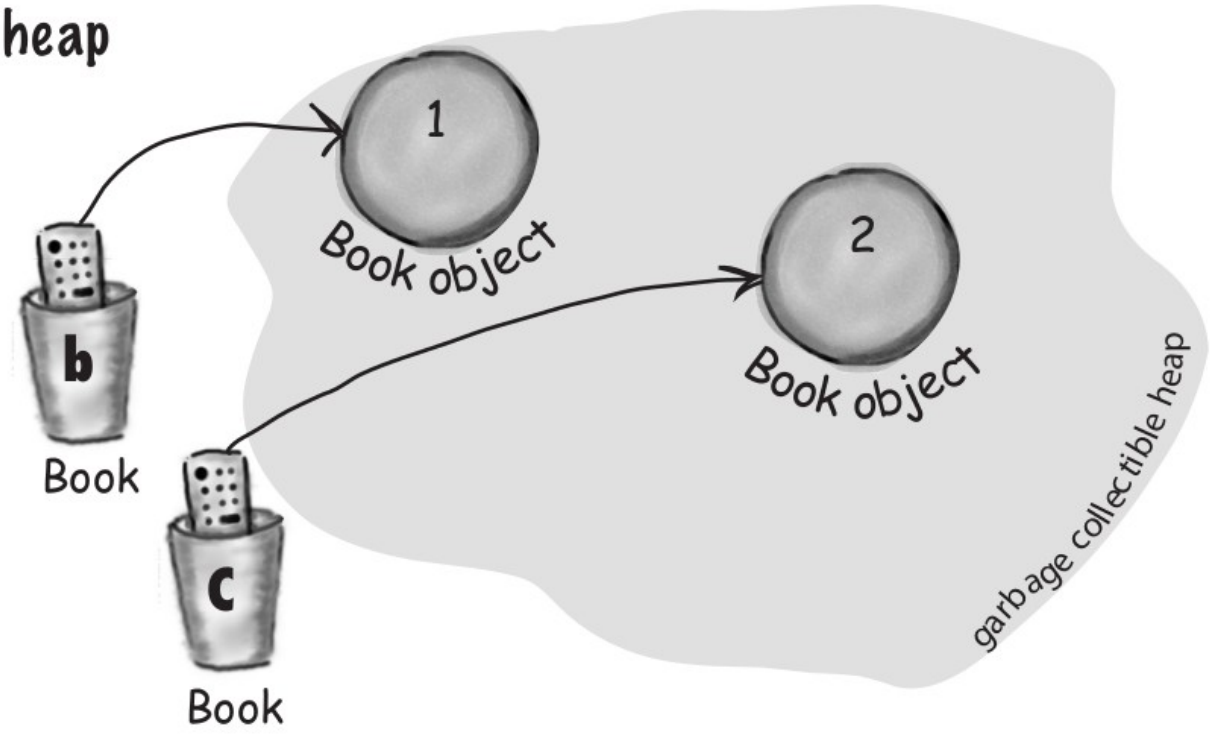
```
Book b = new Book();
Book c = new Book();
```

Declare two Book reference variables. Create two new Book objects. Assign the Book objects to the reference variables.

The two Book objects are now living on the heap.

References: 2

Objects: 2



```
Book d = c;
```

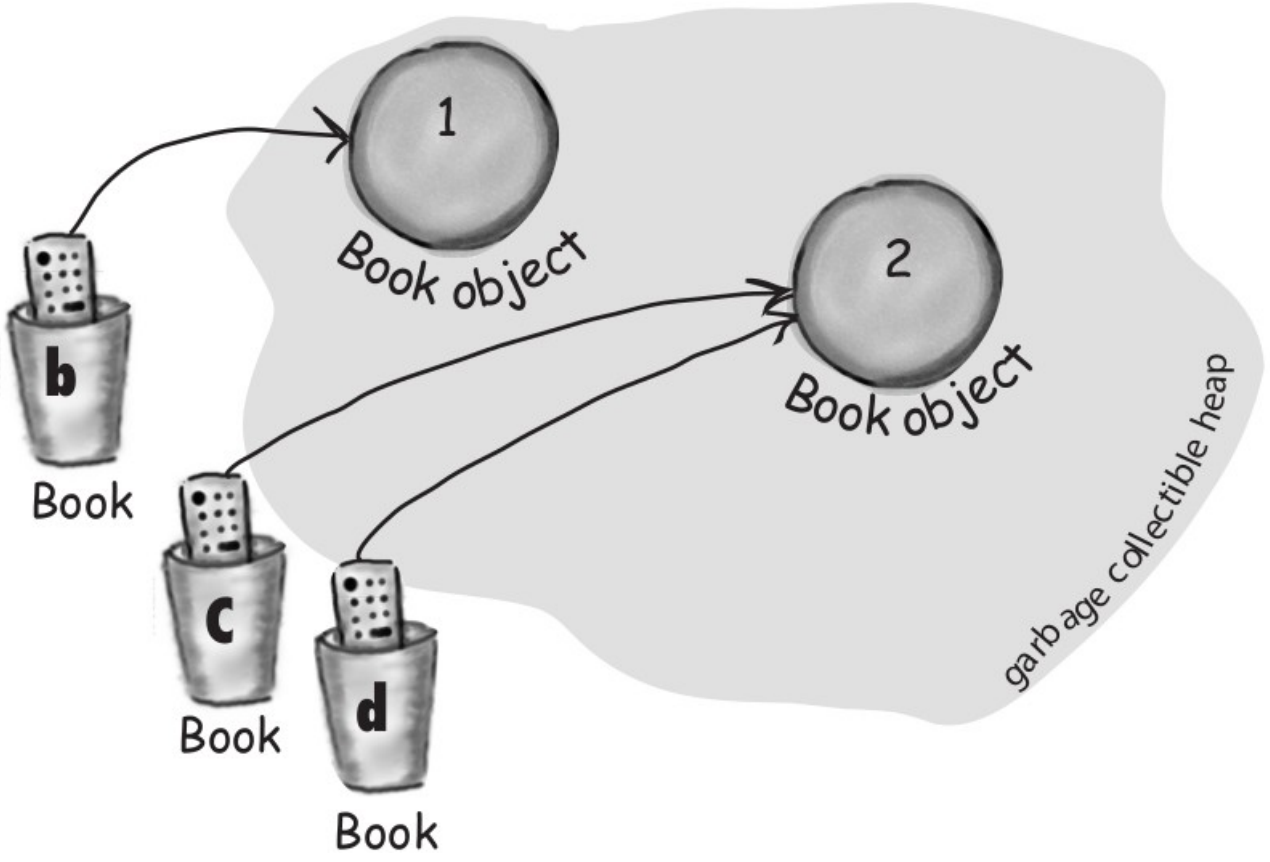
Declare a new Book reference variable. Rather than creating a new, third Book object, assign the value of variable ***c*** to variable ***d***. But what does this mean? It's like saying "Take the bits in ***c***, make a copy of them, and stick that copy into ***d***."

Both *c* and *d* refer to the same object.

The *c* and *d* variables hold two different copies of the same value. Two remotes programmed to one TV.

References: 3

Objects: 2



```
c = b;
```

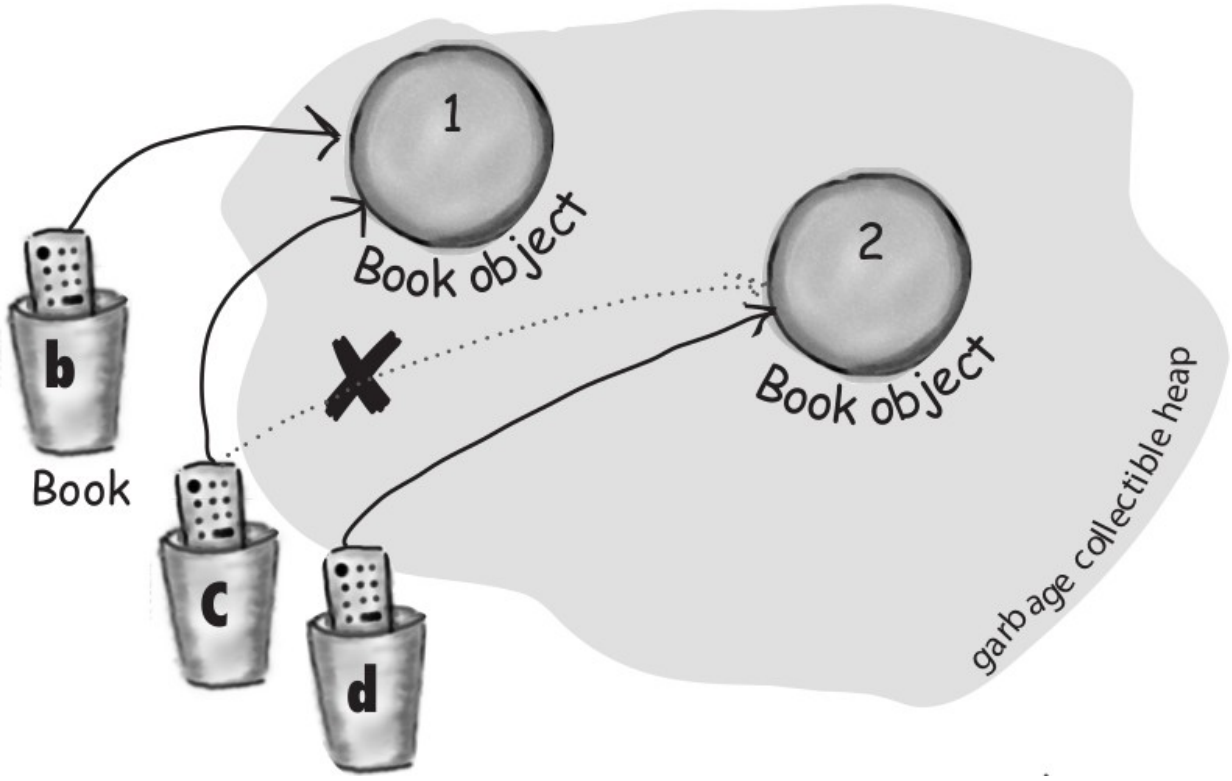
Assign the value of variable ***b*** to variable ***c***. By now you know what this means. The bits inside variable ***b*** are copied, and that new copy is stuffed into variable ***c***.

Both b and c refer to the same object.

The c variable no longer refers to its old Book object.

References: 3

Objects: 2



```
b = c;
```

Assign the value of variable ***c*** to variable ***b***. The bits inside variable ***c*** are copied, and that new copy is stuffed into variable ***b***. Both variables hold identical values.

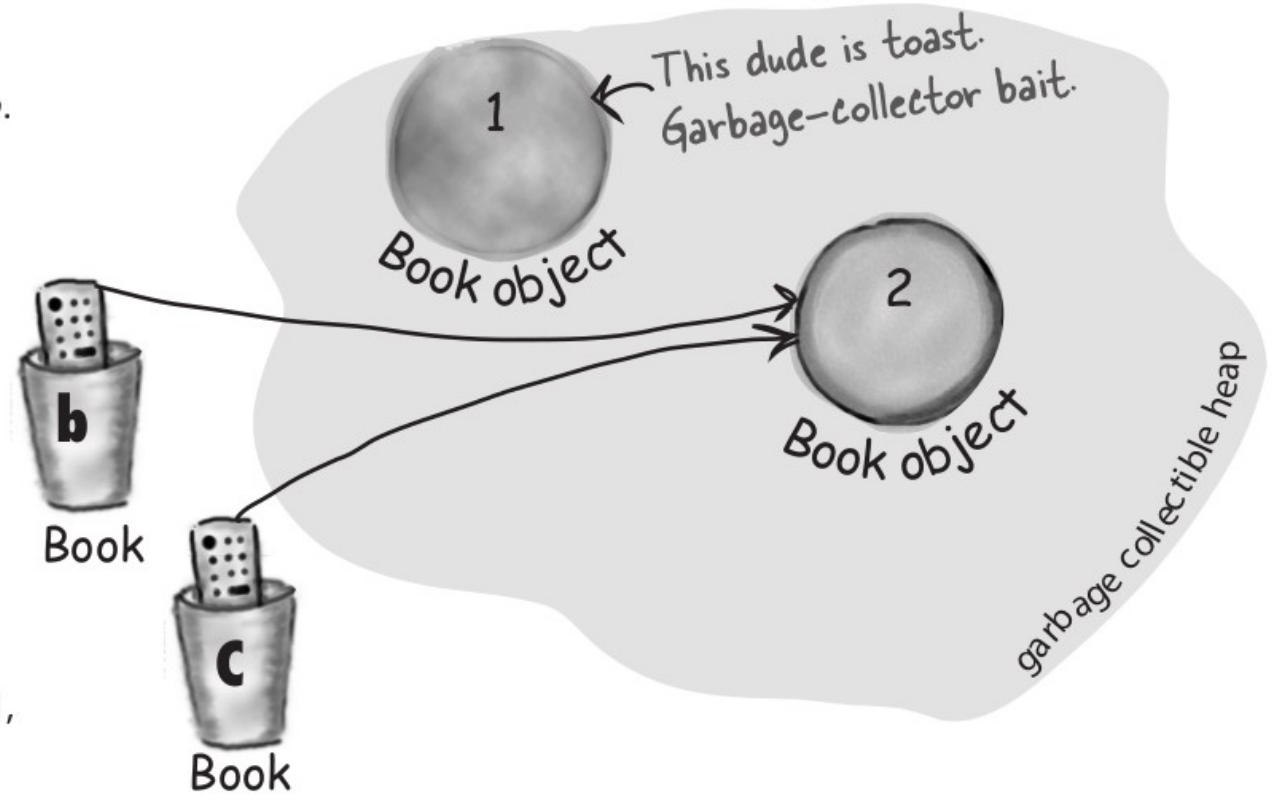
Both b and c refer to the same object. Object 1 is abandoned and eligible for Garbage Collection (GC).

Active References: 2

Reachable Objects: 1

Abandoned Objects: 1

The first object that ***b*** referenced, Object 1, has no more references. It's *unreachable*.



```
c = null;
```

Assign the value `null` to variable ***c***. This makes ***c*** a *null reference*, meaning it doesn't refer to anything. But it's still a reference variable, and another Book object can still be assigned to it.

Object 2 still has an active reference (b), and as long as it does, the object is not eligible for GC.

Active References: 1

null References: 1

Reachable Objects: 1

Abandoned Objects: 1